

Báo cáo rà soát test code

<!-- framework: JUnit | run_id: run_20260617_084126_806737 -->

Framework: JUnit | Run ID: run_20260617_084126_806737

Tóm tắt kỹ thuật

- Framework: JUnit 5 (JUnit Jupiter)
- Kết quả chạy: Tất cả 20 test đều pass, không lỗi biên dịch hay runtime.
- Coverage: 100 % (Jacoco) – bao phủ toàn bộ lớp BankAccount.
- Vấn đề thực thi chính: Không có lỗi ngăn chạy, nhưng thiếu một số test bắt buộc theo kế hoạch kiểm thử (các test kiểu *clarification* và các yêu cầu về @TempDir / factory method).
- Artifact đã xem: BankAccount.java (source) và BankAccountTest.java (test).

Lỗi nghiêm trọng

- Không phát hiện.

Test còn thiếu

- TC-012 – Xác định phương thức công khai nào gọi checkActive().
- Bằng chứng: Kế hoạch kiểm thử yêu cầu một test “clarification” để làm rõ API công khai (ví dụ withdraw, transfer, deactivate) mà gọi checkActive(). Trong BankAccountTest.java không có test nào chỉ mục tiêu này; các test hiện có chỉ kiểm tra hành vi thông qua các phương thức, nhưng không có test riêng để khẳng định mối quan hệ.
- Ảnh hưởng: Không đáp ứng đầy đủ yêu cầu “clarification” trong test plan, gây thiếu tài liệu kiểm thử cho việc bảo trì và mở rộng.
- Cách sửa: Thêm một test, ví dụ:

@Test

```
void test_withdraw_calls_checkActive_when_inactive() {  
    BankAccount acc = new BankAccount("ACC123", 100);  
    acc.deactivate();  
    IllegalStateException ex = assertThrows(IllegalStateException.class,  
    () -> acc.withdraw(10));  
    assertEquals("Account is inactive", ex.getMessage());  
}
```

(Tương tự cho deposit, transfer nếu muốn.)

- TC-013 – Xác nhận sự tồn tại và chữ ký của factory method hoặc constructor cho implicit argument conversion.
- Bằng chứng: Kế hoạch yêu cầu xác nhận có factory method/static hoặc constructor nhận String. Mã nguồn BankAccount chỉ có constructor (String, double); không có factory method nhận duy nhất một String. Test hiện không có bất kỳ @ParameterizedTest nào sử dụng conversion này.
- Ảnh hưởng: Yêu cầu “clarification” không được thực hiện; nếu trong tương lai muốn dùng @ParameterizedTest sẽ thiếu thông tin.
- Cách sửa: Thêm một test (hoặc tài liệu) để xác nhận không có factory method phù hợp, hoặc nếu muốn tạo một factory method mới, triển khai trong lớp và viết test xác nhận. Ví dụ:

@Test

```
void test_no_string_factory_method_exists() {  
    // Kiểm tra phản chiếu (reflection) không tìm thấy method static String->BankAccount  
    assertTrue(Arrays.stream(BankAccount.class.getDeclaredMethods())  
    .noneMatch(m -> Modifier.isStatic(m.getModifiers()))
```

```
&& m.getParameterCount() == 1
&& m.getParameterTypes()[0] == String.class
&& m.getReturnType() == BankAccount.class));
}
```

- TC-014 - Xác nhận việc sử dụng @TempDir static Path chia sẻ giữa các test method.
- Bằng chứng: Kế hoạch để cập tới việc dùng @TempDir static Path để chia sẻ thư mục tạm, nhưng trong BankAccountTest.java không có bất kỳ trường hoặc tham số nào được anotate bằng @TempDir.
- Ảnh hưởng: Không kiểm chứng được yêu cầu “shared temporary directory”, dẫn tới thiếu kiểm thử cho các trường hợp có tương tác file (nếu có).
- Cách sửa: Thêm một trường static được anotate và một test mẫu sử dụng nó, ví dụ:

```
@TempDir
static Path sharedTempDir;

@Test
void test_using_shared_temp_dir() throws IOException {
    Path file = sharedTempDir.resolve("test.txt");
    Files.writeString(file, "data");
    assertTrue(Files.exists(file));
}
```

Assertion yếu

- Không phát hiện.

Rủi ro maintainability

- Không phát hiện.

Gợi ý sửa ngay

1. Thêm các test “clarification” (TC-012, TC-013, TC-014) để đáp ứng đầy đủ test plan.
2. Nếu không cần @TempDir, loại bỏ yêu cầu này khỏi test plan để tránh nhầm lẫn trong tương lai.
3. Cân nhắc tài liệu hóa (comment hoặc README) về việc không có factory method phù hợp, để các nhà phát triển không phải dò tìm trong code.

Báo cáo này dựa trên mã nguồn và log thực thi được cung cấp, không tạo ra bất kỳ API hay hành vi nào không tồn tại trong source.